

Vanishing Gradient Problem in ANN | Exploding Gradient Problem

➤ Introduction:

When we train deep neural networks, we rely on **backpropagation** to adjust the weights. But sometimes, during this process, **gradients become extremely small (vanish)** or **extremely large (explode)** — making the training unstable or even fail completely.

These are known as the:

- **Vanishing Gradient Problem**
 - **Exploding Gradient Problem**
-

➤ Vanishing Gradient Problem

✓ *What Happens:*

In deep networks, during **backpropagation**, gradients are multiplied layer by layer through derivatives of activation functions.

If those derivatives are **less than 1**, multiplying many of them makes the gradients shrink toward **zero**.

$$\text{Gradient at layer } L = \frac{\partial L}{\partial w_L} = \frac{\partial L}{\partial a_L} \cdot \frac{\partial a_L}{\partial z_L} \cdot \frac{\partial z_L}{\partial w_L}$$

If each derivative ≈ 0.5 and you have 50 layers:

$$(0.5)^{50} = \text{almost zero!}$$

→ The earlier layers (closer to the input) stop learning because their gradients vanish.

✓ *Example:*

If you use **sigmoid** or **tanh**, their derivative is small for large input values:

- Sigmoid: derivative ≤ 0.25
- Tanh: derivative ≤ 1

Over many layers → gradients almost disappear.

✓ *Symptoms:*

- Model stops improving.
 - Weights of early layers barely change.
 - Loss function stagnates.
-

➤ Code Demo (Vanishing Gradient)

Example (simplified in PyTorch-like pseudocode):

```

import torch
import torch.nn as nn

model = nn.Sequential(
    nn.Linear(784, 256),
    nn.Sigmoid(),
    nn.Linear(256, 128),
    nn.Sigmoid(),
    nn.Linear(128, 64),
    nn.Sigmoid(),
    nn.Linear(64, 10)
)

x = torch.randn(1, 784)
output = model(x)
output.backward(torch.ones_like(output))
If you check gradients:
for name, param in model.named_parameters():
    print(name, param.grad.abs().mean())

```

→ You'll see gradients in early layers are **much smaller** than in later ones.

➤ How to Handle Vanishing Gradient

There are several practical solutions to overcome this issue 📌

1. Use ReLU or its Variants

- ReLU (Rectified Linear Unit):

$$f(x) = \max(0, x)$$

Derivative is **1** for $x > 0$, preventing vanishing gradients.

- Variants: Leaky ReLU, ELU, GELU — they keep gradients alive for negative x .

2. Proper Weight Initialization

- Xavier Initialization (for tanh/sigmoid)
- He Initialization (for ReLU)

These keep the variance of outputs and gradients roughly consistent across layers.

3. Batch Normalization

- Normalizes layer inputs during training.
- Keeps values in a stable range → reduces vanishing/exploding risks.

4. Residual Networks (ResNets)

- Introduce **skip connections** that pass gradients directly to earlier layers.
- Makes training very deep networks (100+ layers) possible.

Code Demo (Fix using ReLU)

```
model = nn.Sequential(  
    nn.Linear(784, 256),  
    nn.ReLU(),  
    nn.Linear(256, 128),  
    nn.ReLU(),  
    nn.Linear(128, 64),  
    nn.ReLU(),  
    nn.Linear(64, 10)  
)
```

Now, gradients stay much more stable during backpropagation.

➤ ReLU Activation Function

✓ Why ReLU Helps:

- Gradient is **1** when $x > 0$ (does not vanish).
- Simple and fast to compute.
- Allows faster convergence.

✓ Issue:

- ReLU can lead to **Dead Neurons** (when $x < 0 \rightarrow \text{gradient} = 0$).
→ Fixed with **Leaky ReLU**: keeps a small gradient even when $x < 0$.

➤ Code Demo (Comparing Activations)

```
import torch.nn.functional as F  
  
x = torch.linspace(-5, 5, 100)  
y_sigmoid = torch.sigmoid(x)  
y_relu = F.relu(x)
```

Plot and compare:

- Sigmoid squashes outputs between 0–1 (gradients shrink).
- ReLU grows linearly for positive inputs (gradients survive).

➤ Weight Initialization Techniques

✓ Xavier Initialization:

Used for sigmoid/tanh:

$$Var(W) = \frac{1}{n_{in} + n_{out}}$$

✓ He Initialization:

Used for ReLU:

$$Var(W) = \frac{2}{n_{in}}$$

In PyTorch:

```
nn.init.kaiming_normal_(layer.weight)
```

This ensures gradients neither vanish nor explode in early layers.

➤ Batch Normalization

- Normalizes the input to each layer (mean ≈ 0 , variance ≈ 1).
- Reduces internal covariate shift.
- Makes gradient flow smoother and faster.
- Helps networks converge quickly and stably.

```
nn.BatchNorm1d(128)
```

➤ Residual Network (ResNet)

- Adds **skip connections**:

The input of a layer is added directly to its output $\rightarrow$ gradient can “skip” layers.

$$Y = f(x) + x$$

This helps gradients pass through even in very deep models (like 100+ layers).

➤ Exploding Gradient Problem (Opposite Case)

When gradients **grow too large** (e.g., 10^5 or 10^8), the weights overshoot, and the network becomes unstable.

✓ Why it Happens:

- Derivatives > 1 multiplied repeatedly.
- Improper initialization or high learning rate.

✓ **Fixes:**

- Gradient Clipping:

- `torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)`

- Proper initialization (Xavier/He)
 - Lower learning rate
 - Use normalization layers
-

➤ **Outro**

✓ **Key Takeaways**

Problem	Cause	Symptoms	Fix
Vanishing Gradient	Small derivatives (sigmoid/tanh), deep nets	Slow or no learning	ReLU, He/Xavier init, BatchNorm, Residuals
Exploding Gradient	Large derivatives, bad init	Weights blow up, NaNs	Gradient clipping, smaller LR, normalization

➤ **Summary:**

The vanishing and exploding gradient problems were the biggest challenges in early deep learning — and solving them (through ReLU, initialization, and ResNets) is what made **modern deep networks possible**.
